

ML Challenge 2025: Smart Product Pricing

Solution Report

Team Name: DataVengers

Team Members: Thammisetty Srilekha, Udit Jain, Ujjawal Sharma

Submission Date: October 13, 2025

1. Executive Summary

We implement a **RAM-safe sparse pipeline** to predict product prices using **textual catalog content** plus light **regex-based structured features**. Text is vectorized via **TF-IDF word n-grams (1–2)** with an optional **character TF-IDF (3–5)** branch. Numeric and categorical features (units, hosts, etc.) are standardized/one-hot encoded and concatenated into a large sparse matrix. We train multiple lightweight regressors on $\log(1 + \text{price})$ and **blend OOF predictions via Non-Negative Least Squares (NNLS)**. The pipeline **auto-downscales** when RAM is limited, caches features, and reuses artifacts for fast re-runs.

2. Methodology Overview

2.1 Problem Analysis

We treat pricing as supervised regression on the provided fields, focusing on textual cues (quantities, units, pack sizes) and simple metadata. Efficiency, reproducibility, and rules-compliance (no external data or image downloading) are prioritized.

2.2 Approach

- **Structured features from catalog_content:**
 - Regex extraction: *pack_of*, *value_num*, *unit_raw* $\rightarrow$ normalized *unit_base* in {g, ml, count, UNK} with *unit_mult*.
 - Derived: $\text{value_base} = \text{value_num} \times \text{unit_mult}$, $\text{value_per_pack_base} = \text{value_base} / \text{pack_of}$.
 - Text stats: *txt_len*, *txt_words*, *txt_digits*, *txt_upper*, ratios *txt_digit_ratio*, *txt_upper_ratio*.
- **Light features from image_link:** *img_ext*, *img_has_dim*, *img_host* (top-K with “_other_” bucket), *img_len*.
- **Vectorization and scaling:**
 - **Word TF-IDF** (1–2 grams; default cap WORD_TFIDF=65,536, sublinear_tf, 12).
 - **Char TF-IDF** (3–5 grams; optional via USE_CHAR=1, cap CHAR_TFIDF=16,384).
 - Numeric features scaled with `StandardScaler(with_mean=false)`.
 - Categorical encoded with a **safe OHE**: `OneHotEncoder(handle_unknown='ignore', min_frequency=OHE_M)` when supported, else sparse OHE fallback.
- **Modeling (log-target):**
 - **SGDRegressor (squared_error)**: `alpha=1e-5, l2, max_iter=2000, tol=1e-4`.
 - **SGDRegressor (huber)**: `alpha=5e-5, l2, max_iter=2000, tol=1e-4`.
 - **Ridge** (optional via `USE_RIDGE=1`): `alpha=1.0, solver=lsqr`.
 - **XGBRegressor** (optional, if available and not `SPEED_MODE=ultra`); trained with early-stopping *only* when a validation split is provided.

- **Ensembling:** For each fold, we stack per-model OOF predictions (after $\exp(\cdot) - 1$) and compute NNLS weights. At inference, we train each model on the full train set and blend test predictions with those weights. Robust fallbacks (row/weight sanitization) are included.
- **Validation:** **KFold** with **OOF_FOLDS** (default 5). We log **SMAPE**, **RMSE**, **MAE**, and **R²** to `dataset/cv_metrics_log.csv`.
- **Post-processing:** Blend predictions are **clipped** to $[0.01, Q_{0.997}]$ for stability.

3. Model Architecture

3.1 Diagram

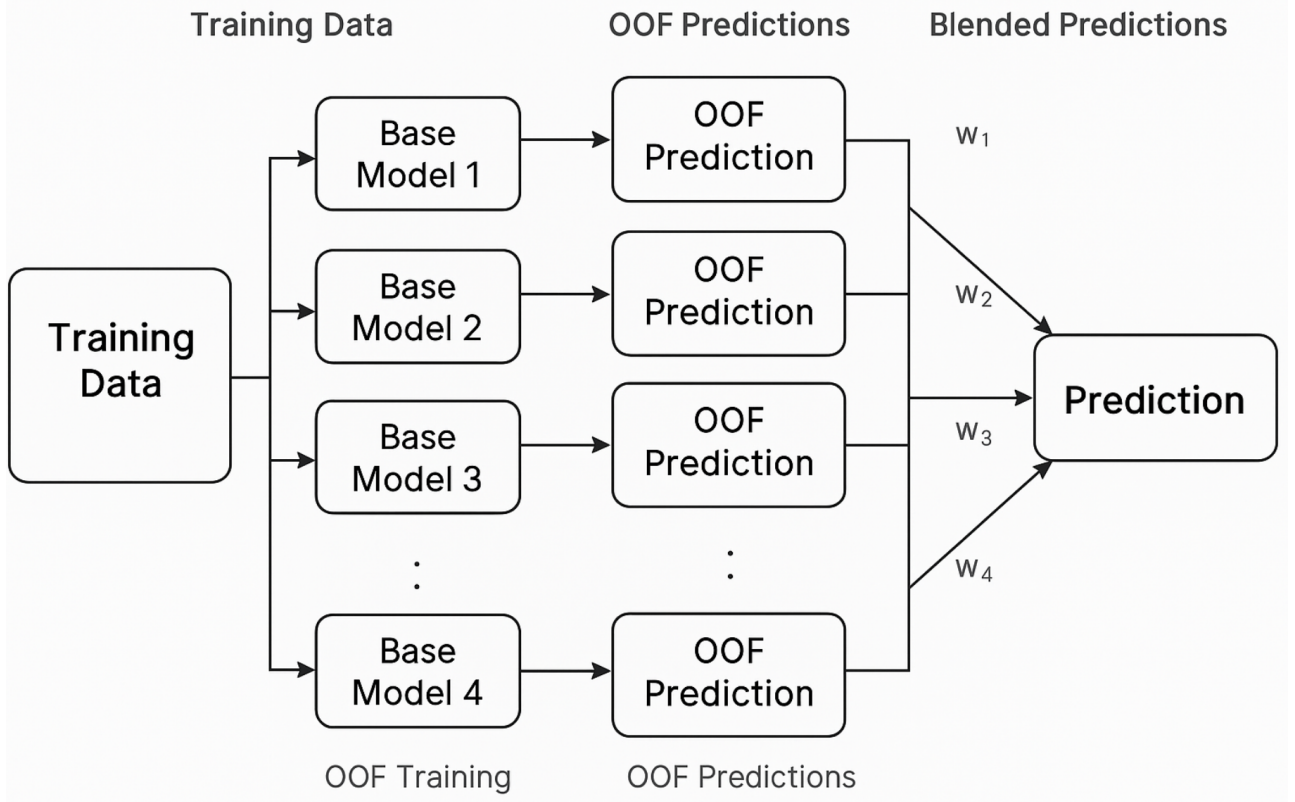


Figure 1: End-to-end pipeline. Text & metadata $\rightarrow$ regex features + TF-IDF (word, optional char) $\rightarrow$ scale/OHE $\rightarrow$ sparse hstack $\rightarrow$ multi-model regressors (SGD Huber, SGD Squared, optional Ridge/XGB) $\rightarrow$ OOF NNLS blend $\rightarrow$ $\exp(\cdot) - 1$ & clipping $\rightarrow$ price.

3.2 Key Implementation Details (mirrors code)

- **Environment switches:** `SPEED_MODE` (ultra|fast|full), `MAX_CHARS`, `WORD_TFIDF`, `CHAR_TFIDF`, `USE_CHAR`, `OOF_FOLDS`, `SEED`, `USE_XGB`, `USE_SGD`, `USE_RIDGE`, `OHE_MIN_FREQ`, `IMG_HOST_TOPK`.
- **RAM-aware downscaling:** if total RAM < 16 GB (via `psutil`), caps are reduced and char TF-IDF may be auto-disabled unless `FORCE_HEAVY=1`.
- **Exact SGD configs:**
 - `sgd_sqr`: `loss='squared_error'`, `alpha=1e-5`, `l2`, `max_iter=2000`, `tol=1e-4`.
 - `sgd_huber`: `loss='huber'`, `alpha=5e-5`, `l2`, `max_iter=2000`, `tol=1e-4`.
- **Ridge (optional):** `alpha=1.0`, `solver='lsqr'`.
- **XGB (optional):** `n_estimators=900`, `learning_rate=0.06`, `max_depth=8`, `subsample=0.8`, `colsample_bytree=0.8`, `min_child_weight=3.0`, `reg_alpha=0.0`, `reg_lambda=1.2`, `tree_method='hist'`, `n_jobs=-1`, `eval_metric='rmse'`.

`random_state=SEED`; early stopping only if a val set is provided.

- **Blending:** NNLS on stacked OOF predictions (fallback to nonnegative least squares via `lstsq` if needed); weights normalized to sum to 1.
- **Caching:** Sparse matrices stored under a deterministic tag: `tfidf_w{WORD}_c{CHAR}_mc{MAX}_useC{0/1}_top`

4. Dataset, I/O and Compliance

4.1 Dataset Columns

- `sample_id` (test, output key)
- `catalog_content` (train & test)
- `image_link` (train & test; used only for lightweight parsing)
- `price` (train target)

4.2 Outputs and Artifacts

- Predictions: `dataset/test_out.csv` with columns `{sample_id, price}`.
- CV metrics log: `dataset/cv_metrics_log.csv`.
- OOF & blend info: `dataset/oof_preds.pkl` (contains per-model OOF arrays and `blend_info` with weights).
- Feature caches: `dataset/{tag}_Xtr.npz`, `dataset/{tag}_Xte.npz`, `dataset/{tag}_meta.pkl`.

4.3 Compliance

- No external data, no web requests, no image downloads.
- Only the provided `train.csv` and `test.csv` are used.
- Deterministic seeds and local caching ensure reproducibility.

5. Training Objective and Metrics

Objective: minimize MSE on $\log(1 + \text{price})$.

Reported metrics: SMAPE, RMSE, MAE, R^2 per fold (logged to CSV).

$$\text{SMAPE} = 100 \times \frac{1}{N} \sum_{i=1}^N \frac{|\hat{y}_i - y_i|}{(|\hat{y}_i| + |y_i|)/2}.$$

6. Results (K-Fold OOF)

Metric	SMAPE (%)	RMSE	MAE	R^2
OOF Mean	73.582	34.529	16.168	-0.0746

7. Conclusion

We present a **sparse, RAM-aware, and cache-friendly** regression pipeline that blends fast linear models (and optional XGB) using **NNLS** over OOF predictions. The design emphasizes reproducibility, graceful degradation on low-RAM systems, and strict data-compliance—yielding a strong, stable baseline for smart product pricing.